Laboratório Funcional · Aula 12

Enzo Seiti Hashimoto · RA 241903572 · Turma esoft6c
Gerado em 16/10/2026, 20:56:47 · tempo registrado no cronômetro: 0:00

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

1- agora(): É impura por se basear na data e hora atual que podem acabar mudando mesmo com os parâmetros.
2-registra(): É impura por usar um print(), causando efeitos colaterais na hora de exibir na tela.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

Linha 1: A-1, A-2
Linha 4: 14
Linha 7: [7, 3, 9] e [3, 7, 9]

Saídas (original e com compor(g, f))

```
(f, g):
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]

(g, f):
1: A-1 A-2
2: True
```

```
3: 8
4: 18
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

Pois sua função utiliza a variável global contador, que é alterada a cada chamada. Isso acaba por dificultar os testes devido a mesma entrada produzir resultados diferentes, sendo assim impossível de prever resultados apenas por olhar os parâmetros.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

(f, g): primeiro o g(4): 3 x 4 =12, depois f(12): 12 + 2 = 14.
(g, f): primeiro o f(4): 4 + 2 = 6, depois o g(6): 3 x 6 = 18.
Resultados são 14 e 18, ou seja, a ordem das variáveis altera o produto.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

O sorted(notas) cria uma nova lista ordenada, mantendo a lista original sem alteração.
O notas.sort() modifica a própria lista. sorted(notas) segue melhor o estilo funcional por tentar evitar a alteração do estado do objeto original.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

(B C)

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

```
(com)
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B

(sem ')
22
14
25
8
negativo
```

```
Backtrace:
           5 (primitive-load "/box/main.scm")
In ice-9/eval.scm:
   191:35  4 (_ #f)
   191:35  3 (_ #f)
   196:27  2 (_ #f)
   223:20  1 (proc #<directory (guile-user) c6b30be4c80>)
In unknown file:
           0 (%resolve-variable (7 . A) #<directory (guile-user) c6b…>)

ERROR: In procedure %resolve-variable:
Unbound variable: A
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo previne que o Scheme leia e execute a lista como uma expressão. Assim, '(A B C) é tratada como uma lista de dados, já o sem o apóstrofo, o Scheme tenta interpretar A como uma função e executar (A B C), causando um erro por A não ser uma função definida.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
C
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

não respondido

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

Vai funcionar, porém em python vai ocorrer erro.

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

```
500
Traceback (most recent call last):
  File "/box/main.py", line 9, in <module>
    print(conta_ate(1_000_000, 0))
          ^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
```

```
            ^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na comum cresce a pilha de execução, isso se deve ao fato de que cada chamada deve esperar o resultado da próxima chamada, já na cauda nada da comum precisa acontecer, já que, a chamada recursiva é a ultima operação e o registro de operação pode ser reutilizado.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme é capaz de executar um milhão de chamadas por aplicar otimização de chamada de cauda, fazendo o reaproveitamento de espaço da pilha. Python infelizmente não faz essa otimização, possuindo um limite de profundidade de recursão e assim entregando RecursionError.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
Tamanho: 5
Soma dos quadrados: 55
Pares: (2 4 6)
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
tamanho: 0  (esperado 3)
soma-quadrados: 0  (esperado 14)
pares: ()  (esperado (2 4 6))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais difícil foi entender a divisão da lista entre car e cdr e perceber que a função precisa chamar a si mesma usando o restante da lista. Em tamanho, null? verifica o fim e cdr percorre a lista. Em soma-quadrados, car pega o elemento atual e cdr continua a recursão. Em pares, car verifica o elemento com even?, enquanto cons monta uma nova lista apenas com os números pares.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

take 5 squares: [0,1,4,9,16]

primeiro 42 (div 1 0): 42

Saídas (original e com member 17)

```
Original:
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
```

```
42

Member 17:
Time limit exceeded
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

Haskell é capaz de imprimir parte de uma lista infinita por utilizar avaliações preguiçosas, calculando os elementos que são necessários, o take 5 square, por exemplo, pede apenas os 5 primeiros elementos, no caso de primeiro 42 (div 1 0), a função primeiro ignora o segundo parâmetro, então o Haskell não necessita calcular div 1 0, portanto não ocorre divisão por zero.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

Member 17 squares nunca termina por não estar entre os numero da lista de quadrados, a função portanto continua procurando na lista infinita. Diferente do member 16 squares termina em 16, como está em lista ordenada e crescente, a função pode ser corrigida e retornar False assim que encontrar o número maior que o valor procurado.

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5) = 15
[1, 2, 3].map(vezes(5)) = 5, 10, 15

Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

somador(10) devolve uma função que mantem referência ao ambiente onde foi criada, essa função por exemplo guarda o valor 10 no seu fechamento (closure), por isso, mesmo depois que somador terminou, mais10(5) ainda consegue usar o 10 e retorna 15.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

compose aplica primeiro o g e depois o f, enquanto isso o andThen aplica primeiro o f e depois o g, assim enquanto compose = 14, andThen = 18. O f.compose(g) corresponde ao compor(f, g) da Prática 1, pois ambos calculam primeiro g(x) e depois f(...).

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
// Prática 7 - Exercício 1: reescreva o laço como um pipeline (stream)
// Cole em https://onecompiler.com/java
import java.util.Arrays;
import java.util.List;

public class Main {
    public static void main(String[] args) {

        List<Integer> nums = Arrays.asList(8, 12, 6);

        int soma = nums.stream()
                .filter(n -> n % 2 == 0)
                .mapToInt(n -> n * n)
                .sum();

        System.out.println("Resultado: " + soma);
    }
}
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
// Prática 7 - Exercício 2: reescreva o laço com filter e map
// Cole em https://onecompiler.com/nodejs
const alunos = [
    { nome: "Ana", nota: 8 },
    { nome: "Bruno", nota: 5 },
    { nome: "Carla", nota: 7 },
    { nome: "Davi", nota: 9 }
];

const aprovados = alunos
    .filter((aluno) => aluno.nota >= 6)
    .map((aluno) => aluno.nome.toUpperCase());

console.log(aprovados.join(", "));
```

Saídas das suas versões

```
1- Resultado: 244
2- ANA, CARLA, DAVI
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A funcional é mais fácil de ler por mostrar diretamente as etapas do processamento, como filtrar, transformar e somar. A de laços no entanto é mais fácil de depurar, por pode acompanhar valores intermediários passo a passo.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

Haskell tornou-se mais fácil de entender por mostrar diretamente as operações de: gerar listas, elevar valores ao cubo e soma-los. Nos que eu aceitaria custos de execução seria para sistemas de grande clareza e facilidade de manutenção, para sistemas de tempo real ou que exigem alto desempenho, eu preferia soluções mais eficientes.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

Notar que a programação funcional não está limitada a linguagens como Haskell e Scheme e Java e JS também possuem recursos que permitem escrever código seguindo essa ideia.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

o erro esta no (b <- h), pois h é apenas um elemento e não uma lista. Podemos corrigir substituindo: [b | b <- t, b <- h], por: [b | b <- t, b <= h. O output que é entregue foi: [2,2,4,7,8,9]